

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	ITA05	Course Title:	COMPUTER VISION
CO Assessed:	CO3 – Precision (BL4)	Assessment:	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	Total Marks:	50 (5 Questions × 10 Mark)
CO1 Statement:	<i>Analyze and extract image features using detection and matching techniques for effective image representation and comparison.(BL4).</i>		
Student Name:	Y.Thanushma	Reg. No.:	192425355
Section / Batch:	Hostel/2024/slot c	Date:	

Code of Conduct

I _Y.Thanushma(192425355)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ Y.Thanushma _____

Instructions

- This test contains 5 questions, each carrying 10 marks. Total: 50 Marks.
- No negative marking. Unanswered questions carry zero marks.
- No calculators or electronic devices permitted.
- Duration: 60 minutes.

Section / Topic	Focus	Q Nos	Marks
Types of Image Processing	Point, Neighborhood, Geometric Processing, Applications	1	10
Image Representation	Grayscale, Binary, Color Representation	2	10
Hough Transform	Line Detection, Circle Detection, Parameter Space	3	10
Scale Invariant Feature Matching (SIFT)	Keypoint Detection, Feature Matching, Scale & Rotation Invariance	4	10
Principal Component Analysis (PCA)	Dimensionality Reduction, Feature Selection, Data Representation	5	10
GRAND TOTAL:		50 Marks	

Annexure A – Experiments

1.Feature Descriptor Comparison (SIFT vs Harris)

Design and perform an experiment to compare Harris Corner Detection and SIFT feature extraction on the same set of images. Explain the theoretical principles behind corner detection and feature descriptors. Implement both techniques using suitable software/tools, document the detected features systematically, and analyze their performance in terms of feature quality, robustness, repeatability, and suitability for image matching applications.

2.Image Enhancement for Feature Detection

Conduct an experiment to evaluate the effect of image enhancement techniques such as histogram equalization and contrast adjustment on feature detection performance. Explain the importance of image enhancement in computer vision. Apply enhancement methods followed by feature detection using appropriate tools, document the intermediate and final outputs clearly, and analyze how enhancement influences feature visibility, detection accuracy, and reliability.

3.Multi-Scale Feature Detection Analysis

Design and implement an experiment to study feature detection at different image scales. Explain the concept of image scaling and scale-space representation in feature extraction. Apply feature detection methods on images of varying resolutions using suitable software/tools, document the detected features systematically, and analyze the effect of scale variations on feature stability, accuracy, and computational performance.

4.Comparative Analysis of Image Filtering Techniques

Conduct an experiment to compare different image filtering techniques, such as Mean, Gaussian, and Median filtering, for preprocessing. Explain the theoretical concepts and objectives of each filtering method. Implement the filters using appropriate tools/software, document the processed images and observations clearly, and analyze their effectiveness in noise reduction while preserving important image features for subsequent feature detection.

5.Complete Object Detection Workflow

Design and perform an experiment to develop an object detection workflow incorporating image preprocessing, edge detection, feature extraction, and shape detection. Explain the purpose and interaction of each processing stage in the workflow. Implement the complete pipeline using suitable software/tools, document the outputs at every stage systematically, and analyze the effectiveness of the integrated approach in achieving accurate and reliable object detection under different image conditions.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts	
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation	
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools	
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results	
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation	

Faculty In-charge
Signature: _____
Date: _____

1.Feature Descriptor Comparison (SIFT vs Harris)

Aim:To compare Harris Corner Detection and SIFT feature extraction on the same image and evaluate their performance in detecting features for image processing applications.

Procedure

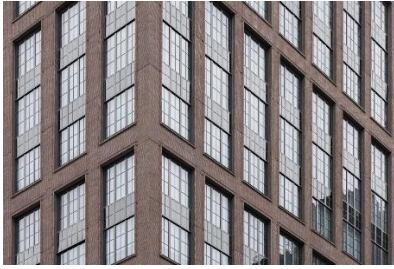
1. Load the input image
2. Convert the image to grayscale
3. Apply Harris Corner Detection to find corner points
4. Mark detected corners on the image
5. Apply SIFT algorithm to detect keypoints and descriptors
6. Draw SIFT keypoints on the image
7. Count number of features detected in both methods
8. Compare results visually and numerically.

Code:

```
import cv2
import numpy as np
img = cv2.imread('sampleinput.jpg')
if img is None:
    print("Error: Image not found!")
    exit()
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray_float = np.float32(gray)
harris = cv2.cornerHarris(gray_float, 2, 3, 0.04)
harris_img = img.copy()
harris_img[harris > 0.01 * harris.max()] = [0, 0, 255]
sift = cv2.SIFT_create()
keypoints, descriptors = sift.detectAndCompute(gray, None)
sift_img = cv2.drawKeypoints(img, keypoints, None)
cv2.imshow("Harris", harris_img)
cv2.imshow("SIFT", sift_img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

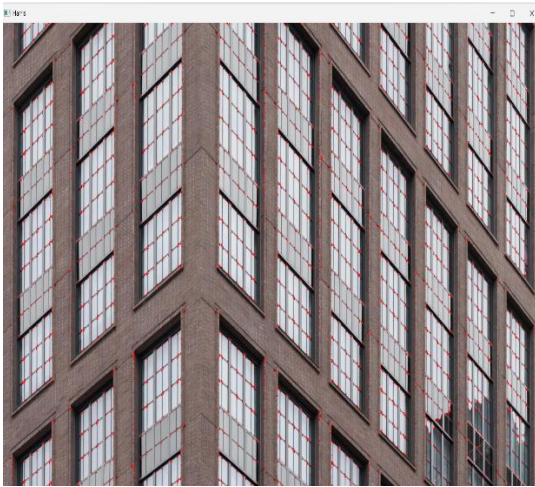
```
print("Harris corners:", np.sum(harris > 0.01 * harris.max()))  
print("SIFT keypoints:", len(keypoints))
```

Sample Input:

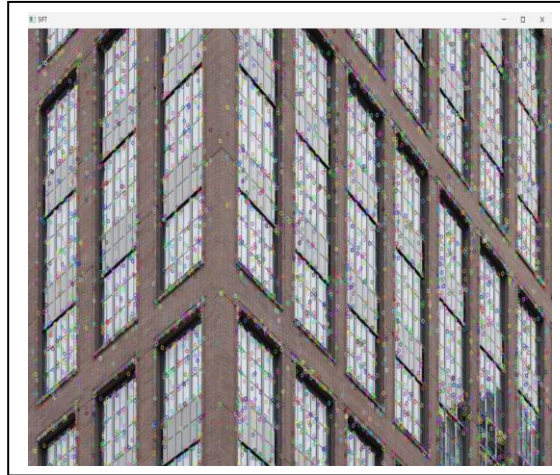


Harris corners: 18778
SIFT keypoints: 6372

Sample Output:



Harris Output



SIFT Output

2. Image Enhancement for Feature Detection

Aim: To study the effect of image enhancement techniques (Histogram Equalization and Contrast Adjustment) on feature detection performance.

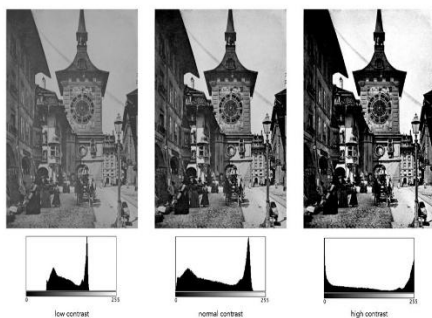
Procedure

1. Load the input image using an appropriate tool (e.g., OpenCV)
2. Convert the image into grayscale format
3. Apply Histogram Equalization to enhance global contrast
4. Apply Contrast Adjustment to improve brightness and intensity variation
5. Perform feature detection (SIFT/Harris) on the original image
6. Perform feature detection on the enhanced images
7. Visualize the detected features for all cases
8. Compare the number and quality of detected features
9. Analyze the effect of enhancement on feature visibility and detection performance

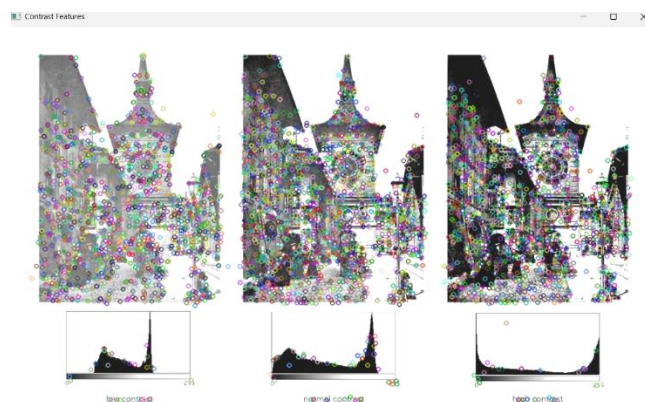
Code:

```
File Edit Format Run Options Window Help
import cv2
import numpy as np
img = cv2.imread('sampleinput2.jpg')
if img is None:
    print("Error: Image not found!")
    exit()
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
hist_eq = cv2.equalizeHist(gray)
alpha = 1.5
beta = 30
contrast_img = cv2.convertScaleAbs(gray, alpha=alpha, beta=beta)
sift = cv2.SIFT_create()
kp1, _ = sift.detectAndCompute(gray, None)
kp2, _ = sift.detectAndCompute(hist_eq, None)
kp3, _ = sift.detectAndCompute(contrast_img, None)
orig_kp = cv2.drawKeypoints(img, kp1, None)
hist_kp = cv2.drawKeypoints(hist_eq, kp2, None)
cont_kp = cv2.drawKeypoints(contrast_img, kp3, None)
cv2.imshow("Original Features", orig_kp)
cv2.imshow("Histogram Equalized Features", hist_kp)
cv2.imshow("Contrast Features", cont_kp)
cv2.waitKey(0)
cv2.destroyAllWindows()
print("Original:", len(kp1))
print("Histogram Equalized:", len(kp2))
print("Contrast Adjusted:", len(kp3))
```

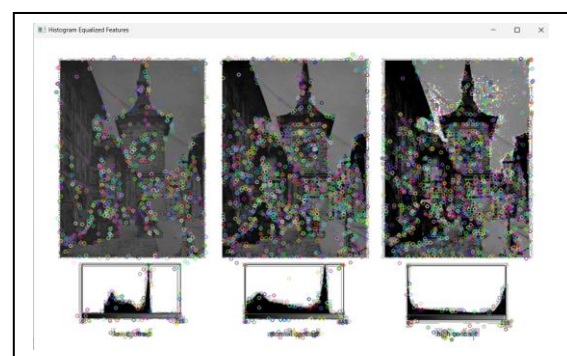
Sample Input:



Output:



Contrast Features



Histogram Equalized

3.Multi-Scale Feature Detection Analysis

Aim: To design and implement an experiment to analyze how feature detection behaves at different image scales, and evaluate its effect on feature stability, accuracy, and computational performance.

Procedure:

1. Load input image.
2. Convert to grayscale.
- Generate multiple scaled versions:**
3. 100% (original)
4. 75%
5. 50%
6. 25%
7. Apply feature detection (SIFT or Harris) on each scale.

Record:

8. Number of detected features
9. Detection time
10. Visualize features at each scale.
11. Compare performance across scales.

Code:

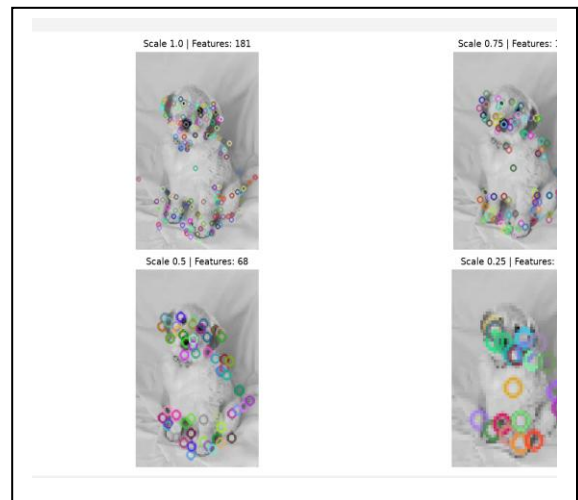
```
sampleinput3.py - C:/Users/Thanushma/Desktop/CV experiments/sampleinput3.py (3.14.3)
File Edit Format Run Options Window Help

import cv2
import time
import matplotlib.pyplot as plt
image = cv2.imread("sampleinput3.jpg")
if image is None:
    print("Error: Image not found")
    exit()
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
scales = [1.0, 0.75, 0.5, 0.25]
sift = cv2.SIFT_create()
results = []
plt.figure(figsize=(12,8))
for i, scale in enumerate(scales):
    resized = cv2.resize(gray, None, fx=scale, fy=scale)
    start = time.time()
    keypoints, descriptors = sift.detectAndCompute(resized, None)
    end = time.time()
    results.append((scale, len(keypoints), end - start))
    img_kp = cv2.drawKeypoints(resized, keypoints, None)
    plt.subplot(2,2,i+1)
    plt.title(f"Scale {scale} | Features: {len(keypoints)}")
    plt.imshow(img_kp, cmap='gray')
    plt.axis('off')
plt.tight_layout()
plt.show()
print("\n--- Feature Detection Results ---")
for scale, count, t in results:
    print(f"Scale: {scale} | Features: {count} | Time: {t:.4f} sec")
```

Input:



Output:



4.Comparative Analysis of Image Filtering Techniques

Aim: To compare Mean, Gaussian, and Median filtering techniques for image preprocessing and analyze their effectiveness in noise reduction and feature preservation.

Procedure:

- Load the input image
- Convert the image to grayscale
- Apply Mean Filter using a 3×3 or 5×5 kernel
- Apply Gaussian Filter with suitable kernel size and sigma
- Apply Median Filter with appropriate kernel size
- Display the original and filtered images
- Compare the results based on:
 - Noise reduction
 - Edge preservation
 - Smoothness
- Analyze which filter performs better for different noise types

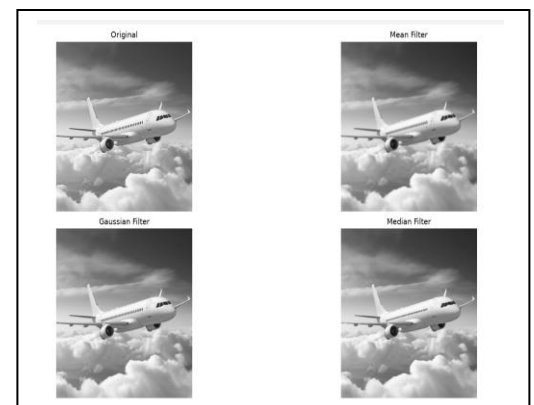
Code:

```
sampleinput4.jpg - C:\Users\manish\My Desktop\CV Experiments\sample
File Edit Format Run Options Window Help
import cv2
import matplotlib.pyplot as plt
image = cv2.imread("sampleinput4.jpg")
if image is None:
    print("Error: Image not found")
    exit()
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
mean = cv2.blur(gray, (5,5))
gaussian = cv2.GaussianBlur(gray, (5,5), 0)
median = cv2.medianBlur(gray, 5)
plt.figure(figsize=(10,8))
plt.subplot(2,2,1)
plt.title("Original")
plt.imshow(gray, cmap='gray')
plt.axis('off')
plt.subplot(2,2,2)
plt.title("Mean Filter")
plt.imshow(mean, cmap='gray')
plt.axis('off')
plt.subplot(2,2,3)
plt.title("Gaussian Filter")
plt.imshow(gaussian, cmap='gray')
plt.axis('off')
plt.subplot(2,2,4)
plt.title("Median Filter")
plt.imshow(median, cmap='gray')
plt.axis('off')
plt.tight_layout()
plt.show()
```

Input:



Output:



5. Complete Object Detection Workflow

Aim: To design and implement a complete object detection workflow using image preprocessing, edge detection, feature extraction, and shape detection, and evaluate its effectiveness under different image conditions.

Procedure:

- Load the input image
- Convert to grayscale
- Apply Gaussian blur for noise reduction
- Perform edge detection using Canny
- Extract features using SIFT
- Detect contours for shape detection
- Draw bounding boxes around detected objects

- Display outputs at each stage

Code:

```
import cv2
import matplotlib.pyplot as plt
image = cv2.imread("sampleinput5.jpg")
if image is None:
    print("Error: Image not found")
    exit()
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5,5), 0)
edges = cv2.Canny(blur, 100, 200)
sift = cv2.SIFT_create()
keypoints, _ = sift.detectAndCompute(gray, None)
feature_img = cv2.drawKeypoints(image, keypoints, None)
contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
shape_img = image.copy()
for cnt in contours:
    x, y, w, h = cv2.boundingRect(cnt)
    if w > 30 and h > 30:
        cv2.rectangle(shape_img, (x, y), (x+w, y+h), (0,255,0), 2)
plt.figure(figsize=(12,10))
plt.subplot(2,3,1)
plt.title("Original")
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.axis('off')
plt.subplot(2,3,2)
plt.title("Grayscale")
plt.imshow(gray, cmap='gray')
plt.axis('off')
plt.subplot(2,3,3)
plt.title("Blur")
plt.imshow(blur, cmap='gray')
plt.axis('off')
plt.subplot(2,3,4)
plt.title("Edges")
plt.imshow(edges, cmap='gray')
plt.axis('off')
plt.subplot(2,3,5)
plt.title("Features")
plt.imshow(cv2.cvtColor(feature_img, cv2.COLOR_BGR2RGB))
plt.axis('off')
plt.subplot(2,3,6)
plt.title("Detected Objects")
plt.imshow(cv2.cvtColor(shape_img, cv2.COLOR_BGR2RGB))
plt.axis('off')
plt.tight_layout()
plt.show()
```

Input:



Output:

